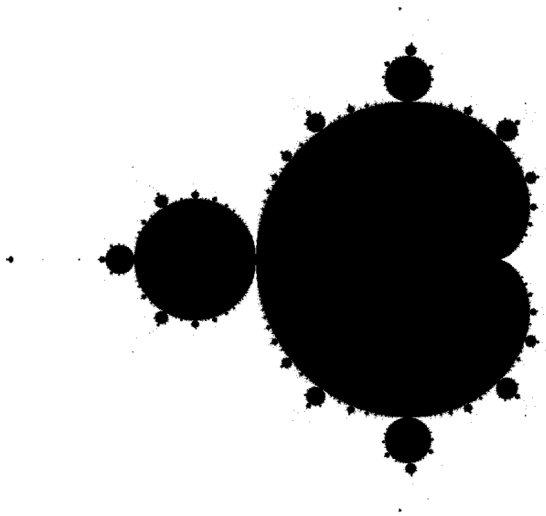
Mandelbrot Explorer

In which we discover a wild landscape made from a simple rule

The Mandelbrot set is one of the most famous objects in mathematics: a fractal generated from a tiny equation. Despite its intricate appearance, the entire image emerges from repeatedly applying a few lines of arithmetic to complex numbers.

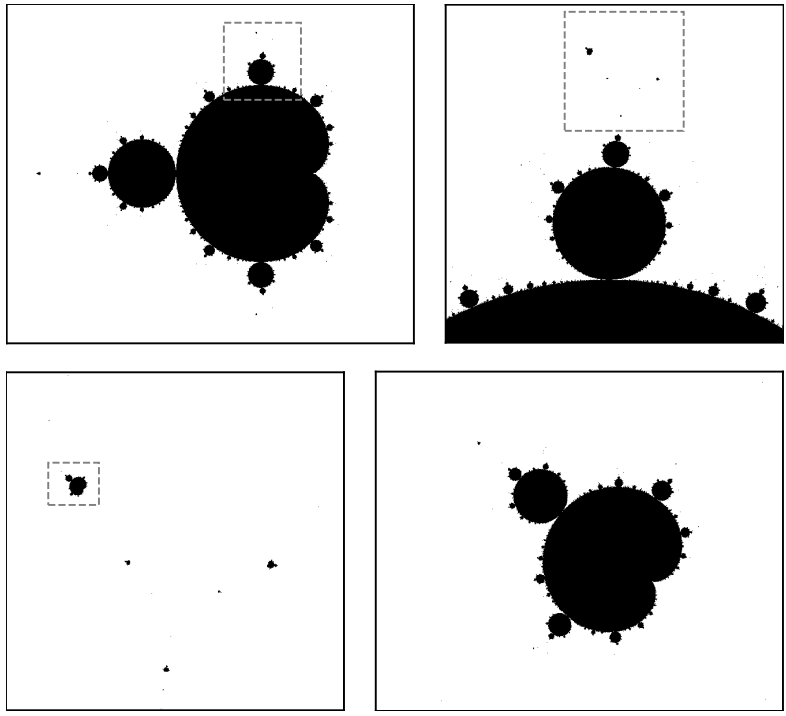


A full view of the Mandelbrot set.

For decades, recreational programmers have written Mandelbrot viewers, or explorers, as a classic demonstration of how simple code can create infinite complexity. The goal of this chapter is to build our own explorer so we can zoom in and navigate through this fascinating set.

The Mandelbrot set is named after Benoît Mandelbrot, a French-American mathematician who first recognized its remarkable fractal structure. Mandelbrot was something of a maverick. While other mathematicians focused on smooth, idealized objects (clean curves, stable systems, and equations that behaved predictably), he was drawn to roughness and irregularity. He studied coastlines, turbulence, and market fluctuations, phenomena often dismissed as noisy, pathological, or not "real mathematics." Where others saw messiness, he recognized hidden structure. In many such phenomena, he noticed patterns that repeat at different scales. He called those patterns *fractals*.

It was not until 1979 that he encountered what many consider the quintessential fractal: the Mandelbrot set. While working at IBM Thomas J. Watson Research Center, he used computers to visualize mathematical formulas that had previously been too tedious to explore by hand. One of those



The Mandelbrot set is an example of a fractal: zooming into the set reveals infinitely many smaller copies of the whole set, called minibrots.

experiments revealed an astonishing discovery: a simple iterative equation that produced a shape of endless complexity, with miniature copies of itself appearing again and again no matter how deeply one zoomed in. What began as a mathematical curiosity soon became one of the most iconic images in science.

The Mandelbrot Set

The Mandelbrot set is based on a simple function applied repeatedly:

$$z \to z^2 + c$$

- Pick a complex number c
- Start with $z = 0$
- Apply the function over and over

The resulting sequence is called the orbit of 0:

$$0, c, c^2 + c, (c^2 + c)^2 + c, \ldots$$

If the orbit of 0 stays bounded, then c is in the Mandelbrot set. If the orbit of 0 escapes to infinity, c is not in the set.

Does 1 belong to the set? The orbit of 0 for $z \to z^2 + 1$ is:

$$0, 1, 2, 5, 26, 677, \ldots$$

The orbit escapes to infinity: 1 is not in the Mandelbrot set.

Does the complex number i belong to the set? The orbit of 0 for $z \to z^2 + i$ is:

$$0, i, -1 + i, -i, -1 + i, \ldots$$

The orbit is cyclic and therefore bounded: i is in the Mandelbrot set.

The Escape-Time Algorithm

The standard way of deciding whether a point c belongs to the Mandelbrot set is a surprisingly low-tech heuristic: compute a fixed number of iterations of the orbit of 0.

If at any time $|z| > 2$, then conclude that c is not in the set. We say that the orbit has *escaped*. It will never return to that disk and, instead, will go to infinity exponentially fast.

If after all the iterations the orbit hasn't escaped, treat the point as probably belonging to the Mandelbrot set. We say *probably* because this algorithm can produce false positives. Some points that are not in the set, especially those close to the boundary, may need more iterations to escape. This typically happens when we zoom into the set. The usual solution is simply to increase the number of iterations.

The escape-time algorithm is described in the following program, in the function `is_in_mandelbrot`. Notice that we iterate at most `MAX_ITERS` times. If the orbit is still bounded after that, we return True.

♦ ASCII Mandelbrot

This program outputs an ASCII rendering of the Mandelbrot set.

► `github.com/n3times/rp2/blob/main/mandelbrot_ascii.py`

```
$ python mandelbrot_ascii.py
```

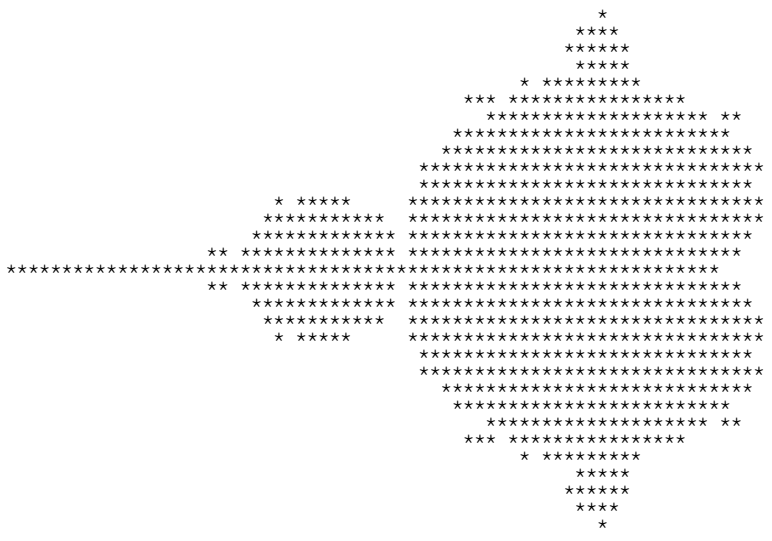
```
# Choose values that produce the same output as the first
# known rendering of the Mandelbrot set.

MAX_ITERS = 200
ROWS, COLS = 31, 71
MIN_X, MAX_X = -1.975, 0.475
MIN_Y, MAX_Y = -0.8715, 0.8715

def is_in_mandelbrot(c):
    z = 0
    for _ in range(MAX_ITERS):
        if abs(z) > 2: return False
        z = z * z + c
    return True

def translate(col, row):
    real = MIN_X + col / (COLS - 1) * (MAX_X - MIN_X)
    imag = MIN_Y + row / (ROWS - 1) * (MAX_Y - MIN_Y)
    return complex(real, imag)

for row in range(ROWS):
    for col in range(COLS):
        c = translate(col, row)
        print("*" if is_in_mandelbrot(c) else " ", end="")
    print()
```



A rendering of the first known representation of the Mandelbrot set.

Though the output is a bit rough, it already shows some of the essential pieces of the set. The large heart-shaped region, on the right side, is known as the *main cardioid*. On the left, there is a thin long antenna and, in between the two, sits a circular region known as the *period-2 bulb*. If we go into more detail, we find smaller bulbs, notably just above and below the main cardioid.

Incidentally, this output matches *exactly* the first known visualization of the set, in 1978, the work of mathematicians Robert W. Brooks and Peter Matelski.

Normally, we would have chosen a more standard view, for example we would have started at `MIN_X = -2`, since $z = -2$ belongs to the Mandelbrot set. This would have produced a slightly different output. But producing the exact original output was a fun challenge we couldn't resist. If one day the original FORTRAN 66 listing resurfaces, it would be interesting to see what exact constants were actually used.

This program is, of course, just a first step and it is a bit clunky. If we want to zoom in or change the number of iterations, we need to modify the program by hand, each time, and run it again. And, at that resolution, we won't be able to explore the finer intricacies of the Mandelbrot set. Finally, color is missing.

Of course, with a budget of 99 lines, and with a touch of Pygame, we can do much better.

Coloring Strategies

Much of what sets the character and aesthetics of a Mandelbrot explorer is the coloring strategy.

Traditionally, the set itself is colored black, and that's what we will do in our program. The interesting question is how to color the points outside the set.

If you look at almost any rendering of the Mandelbrot set, you will see many colors. For example, you may see vivid reds and yellows, and electric whites, typically close to the set, but also greens, cyans, and darker blues a little further away.

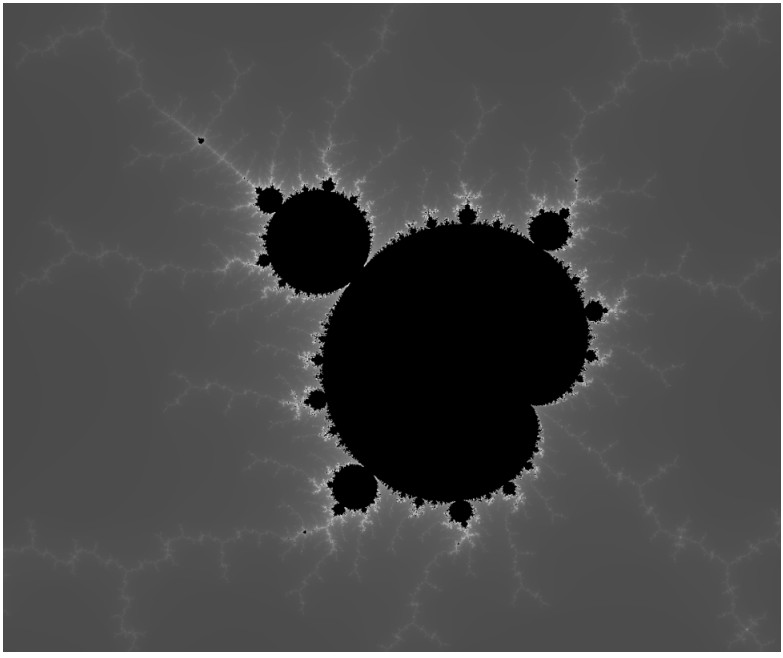
There are actually many possible palettes, and the choice is often an artistic one. The main constraint is to reserve black for the set itself, and avoid dark colors too close to the boundary. The set itself should be clearly visible.

If you want minimalism, you could just color the points outside the set uniformly white. Most of what makes the Mandelbrot set fascinating will

be there as you zoom in. The spirals, the elephant-looking shapes, and all the smaller copies of the whole set, known as minibrots.

If you prefer understated elegance, I recommend a simple monochrome palette. For example, the palette, defined here in code, that starts at white near the set and gradually fades to dark gray farther away:

```
def color_for_iters(iters, max_iters):
    if iters == max_iters: return (0, 0, 0)
    val = 64 + int(192 * iters/max_iters)
    return (val, val, val)
```



A grayscale rendering of a minibrot.

It uses the number of iterations necessary for a point to escape, `iters`, to assign the shade. The ratio `iters/max_iters` serves as a rough indicator of how close the point is to the set and is the classic quantity used for coloring Mandelbrot images.

If `iters == max_iters`, we treat the point as belonging to the set and color it black: (0, 0, 0).

For very small values of iters, meaning points far from the set, we assign a dark gray: (64, 64, 64). As iters increases, and points get closer to the set, the shade moves toward pure white: (255, 255, 255).

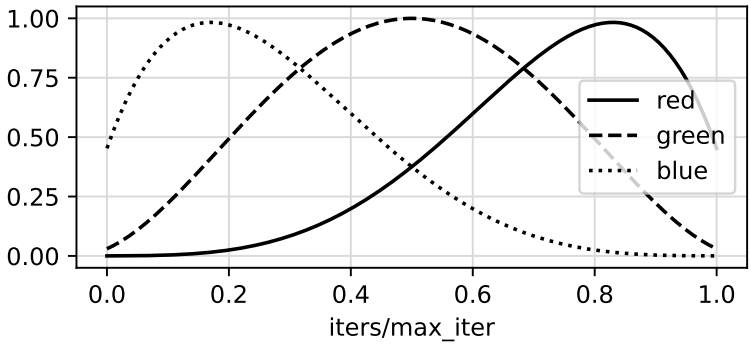
The result is a crisp black set, outlined by a white halo, with rich shading as one zooms in.

The more classic palettes use actual colors, with a wide range that visits much of the spectrum. This is the approach we take for our program. We use:

```
def color_for_iters(iters, max_iters):
    if iters == max_iters: return (0, 0, 0)
    t = (iters/max_iters + 0.05) / 1.1

    # red peaks at t = 0.8, close to the Mandelbrot set
    # blue at t = 0.2 (far away), green at t = 0.5 (in between)
    r = int(255.0 * 12 * (1 - t) * t**4)
    g = int(255.0 * 16 * (1 - t)**2 * t**2)
    b = int(255.0 * 12 * (1 - t)**4 * t)

    return (r, g, b)
```



The coloring strategy used in our program. Each point gets its red/green/blue values as functions of its normalized escape time. Points far from the Mandelbrot set, those with a small escape time, are bluish while points close to the Mandelbrot set are reddish.

This function makes points close to the boundary reddish. Points further from the set are assigned orange, then yellow, green, cyan and finally blue.

To understand why this is the case, note that this function shares much of the same logic with the previous one. As we did earlier, points inside the set are colored black. For the other points, the function considers how close they are to the boundary by using the same ratio `iters/max_iters`. This quantity determines the color (r, g, b).

The mysterious-looking polynomials are chosen so that each color channel "turns on" and "turns off" at different values of t.

For example, the red channel uses $12(1-t)t^4$. As t increases from 0 to 1, this function stays near zero at first, then rises sharply and peaks at $t = 0.8$, before gradually decreasing again.

This means red is strongest for values of t close to 0.8, that is, for points close to the boundary of the Mandelbrot set.

In other words, each color channel is controlled by a smooth "bump" function that turns on, peaks at a specific value of t, and then fades out again.

The affine transformation in the code:

```
t = (iters/max_iters + 0.05) / 1.1
```

shrinks the interval [0, 1] to roughly [0.05, 0.95] and ensures that (r, g, b) is never black outside the set.

If the whole thing feels handcrafted, it is because choosing a palette is more art than science. It depends very much on taste. Trial and error, and tweaks, are part of the process.

We find that other polynomials don't make the set look as good and balanced. And some classic functions, such as those based on cosines, are a little too cumbersome for our goal.

That said, you are encouraged to play with other methods and palettes. This is very much part of the fun of exploring the Mandelbrot set.

Concentric Ring Rendering

Now that we have a coloring strategy, we can render the view. At a basic level, it is just a matter of iterating through all the pixels of the current view, and, for each one:

- Determining the complex number c the pixel maps to.
- Computing the escape time for c.
- Coloring the pixel using the escape time and the coloring strategy.

Rendering a full view is time-consuming and presents a bit of a challenge if we want the UI to remain responsive. For example, the user should be able to move up-and-right by clicking quickly ↑ and →, instead of going up first, waiting for the whole view to render, and only then going right.

The standard solution, and the one we use, is to draw only a few pixels at a time and then check for user input. If the user has panned or zoomed elsewhere, we interrupt the partial rendering and start a new view. Otherwise, we draw a few more pixels and repeat.

We still need to decide *which* pixels to render first. A common technique is to fill the screen top to bottom, or left to right. While this is fine, notice that we are most often interested in the center of the view. This is where our eyes are naturally drawn when zooming and navigating in real life. The solution is to use a *concentric ring algorithm* and start drawing from the center. We draw the pixel at the center first, and then the boundaries of larger and larger squares around it. It is a bit more complex than the traditional scanline, but it is well worth the additional care.

♦ Mandelbrot Explorer

This program allows the user to explore the Mandelbrot set. It uses the following controls:

- Left mouse click : center and zoom in 2x
- Right mouse click : zoom out 2x, keeping the current center
- ← → ↑ ↓ : pan in one of the four directions
- [and] : decrease/increase `max_iters` by 2x
- R : Reset the view

► `github.com/n3times/rp2/blob/main/mandelbrot_explorer.py`

```
$ pip install pygame
$ python mandelbrot_explorer.py
```

```
import pygame, sys

W, H = 800, 600
INITIAL_CENTER = -0.5 + 0j
INITIAL_ZOOM = 1
INITIAL_MAX_ITERS = 128
RANGE_X_FOR_ZOOM_1 = 4

def mandelbrot_iters(c, max_iters):
    z = 0
    for i in range(max_iters):
        if abs(z) > 2: return i
        z = z*z + c
    return max_iters

def pixel_to_complex(pixel, center, zoom):
    x, y = pixel
    dx, dy = x - W//2, H//2 - y
    scale = RANGE_X_FOR_ZOOM_1 / W / zoom
    return center + complex(dx * scale, dy * scale)
```

```
21
22 def color_for_iters(iters, max_iters):
23     if iters == max_iters: return (0, 0, 0)
24     t = (iters/max_iters + 0.05) / 1.1
25
26     # red peaks at t = 0.8, close to the Mandelbrot set
27     # blue at t = 0.2 (far away), green at t = 0.5 (in between)
28     r = int(255.0 * 12 * (1 - t)    * t**4)
29     g = int(255.0 * 16 * (1 - t)**2 * t**2)
30     b = int(255.0 * 12 * (1 - t)**4 * t)
31
32     return (r, g, b)
33
34 def draw_pixel(surface, x, y, center, zoom, max_iters):
35     if not(0 <= x < W and 0 <= y < H): return
36     c = pixel_to_complex((x, y), center, zoom)
37     iters = mandelbrot_iters(c, max_iters)
38     surface.set_at((x, y), color_for_iters(iters, max_iters))
39
40 def draw_ring(surface, ring, center, zoom, max_iters):
41     for y in (H//2 - ring, H//2 + ring):
42         for x in range(W//2 - ring, W//2 + ring + 1):
43             draw_pixel(surface, x, y, center, zoom, max_iters)
44     for y in range(H//2 - ring, H//2 + ring):
45         for x in (W//2 - ring, W//2 + ring):
46             draw_pixel(surface, x, y, center, zoom, max_iters)
47
48 def reset_view():
49     return INITIAL_CENTER, INITIAL_ZOOM, INITIAL_MAX_ITERS, 0
50
51 pygame.init()
52 screen = pygame.display.set_mode((W, H))
53 clock = pygame.time.Clock()
54
55 center, zoom, max_iters, next_ring = reset_view()
56
57 while True:
58     for e in pygame.event.get():
59         if e.type == pygame.QUIT:
60             pygame.quit()
61             sys.exit()
62
63         # Control zoom, panning, iterations, and reset
64         if e.type == pygame.MOUSEBUTTONDOWN:
65             if e.button == pygame.BUTTON_LEFT:
66                 center = pixel_to_complex(e.pos, center, zoom)
67                 zoom *= 2
68             elif e.button == pygame.BUTTON_RIGHT:
69                 zoom = max(1, zoom // 2)
```

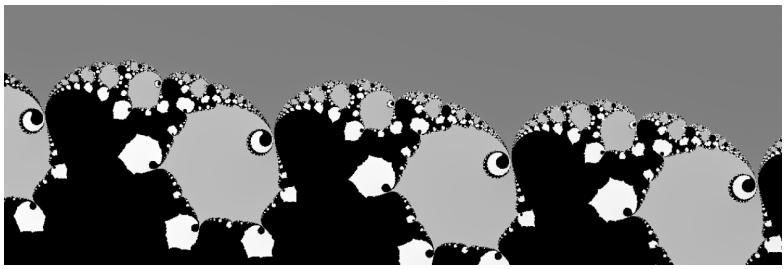
```
            else: continue
            next_ring = 0
        if e.type != pygame.KEYDOWN: continue
        if e.key == pygame.K_LEFTBRACKET:
            max_iters = max(1, max_iters // 2)
        elif e.key == pygame.K_RIGHTBRACKET:
            max_iters *= 2
        elif e.key == pygame.K_LEFT:  center -=  0.5 / zoom
        elif e.key == pygame.K_RIGHT: center +=  0.5 / zoom
        elif e.key == pygame.K_DOWN:  center -= 0.5j / zoom
        elif e.key == pygame.K_UP:    center += 0.5j / zoom
        elif e.key == pygame.K_r:
            center, zoom, max_iters, next_ring = reset_view()
        else: continue
        next_ring = 0

    # Prepare to draw
    if next_ring == 0:
        screen.fill((255, 255, 255))
        pygame.display.set_caption(
            f"{center} -- Zoom {zoom:,} -- Iters {max_iters:,}")

    # Draw a few rings at a time
    if next_ring <= max(W, H) // 2:
        for _ in range(3):
            draw_ring(screen, next_ring, center, zoom, max_iters)
            next_ring += 1

    pygame.display.flip()
    clock.tick(60)
```

In general, you will want to increase `max_iters` gradually as you zoom in: key `]`. Note that higher values come with a trade-off. They slow down the rendering and different values achieve different aesthetics. This is similar to artistic photography where a slight blur is sometimes preferable to a sharp focus.



A deep zoom into Elephant Valley, using MAX_ITERS = 256.

What regions should you visit? A. K. Dewdney wrote for his celebrated August 1985 column in *Scientific American* (“Computer Recreations”):

> Where should one explore the complex plane? Near the Mandelbrot set, of course, but where precisely? Hubbard says that "there are zillions of beautiful spots." Like a tourist in a land of infinite beauty, he bubbles with suggestions about places readers may want to explore. They do not have names like Hawaii or Hong Kong: "Try the area with the real part between .26 and .27 and the imaginary part between 0 and .01."

Dewdney then notes that Hubbard also suggested the regions around −0.75 and around −1.25.

Today, these places are as beautiful as ever, except that now some of them do have names. The region at around 0.26 is known as *Elephant Valley*. The one at around −0.75 is *Seahorse Valley*.

I encourage you to visit them. To Hubbard’s suggestions, I would add −1.75 where you will find a minibrot. Actually, infinitely many of them are hiding around.

Beyond 99 Lines

If you plan to spend lots of time exploring the Mandelbrot set, there are several ideas worth considering.

One easy optimization is to detect, early on, if a point belongs to the main cardioid or to the period-2 bulb. In the escape-time algorithm, these points are expensive because they never escape and therefore use the full `MAX_ITERS` iterations. The good news is that there is no need to iterate at all: there are short analytical tests for those points.

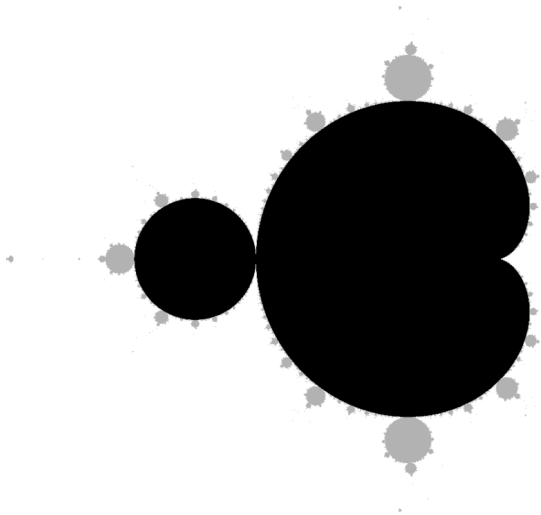
For example, the period-2 bulb is a perfect circle of radius 0.25 centered at -1, and this test will do:

```
def in_period_2_bulb(c):
    return abs(c + 1) <= 0.25
```

The equivalent test for the main cardioid is:

```
def in_main_cardioid(c):
    x, y = c.real, c.imag
    q = (x - 0.25)**2 + y**2
    return q * (q + (x - 0.25)) <= 0.25 * y**2
```

These savings can be substantial if `MAX_ITERS` is large and a significant portion of the period-2 bulb or the cardioid is visible.



Points belonging to the period-2 bulb and the main cardioid (shown in black) can be identified readily, without running the time-consuming escape-time algorithm. They represent over 91% of the Mandelbrot set area.

Another common optimization is to code the escape time condition ($|z| > 2$) as:

```
z.real ** 2 + z.imag ** 2 > 4
```

This avoids the implicit computation of a square root. It obscures the math a little bit, and the speedup is modest on modern machines so this is not a must.

As you explore the set, you may want to save memorable views. Pygame provides this functionality nearly for free. Just add this to the event loop to save the current rendering when the user presses the S key:

```
        elif e.key == pygame.K_s:
            pygame.image.save(screen, "mandelbrot.png")
```

I recommend including additional information in the PNG filename, for example the location and zoom level.

As I mentioned earlier, choosing the right palette and method is lots of fun. You may want to experiment by yourself, but also check what previous explorers have done.

In our program, we used the *escape time* as a very rough proxy for the

distance to the set. This technique produces beautiful images but can result in noticeable color banding. Indeed, the escape time, being an integer, is a rather coarse measure. A variation of the *escape-time algorithm* uses interpolation to compute a more granular *fractional* escape time, resulting in smoother images. Whether this is an improvement depends very much on taste.

Another method, called *distance estimation*, attempts to find a more accurate estimate of the actual distance to the boundary of the set. It is visually less striking but much better at revealing the actual structure of the Mandelbrot set, especially the thin filaments that connect larger regions of the set.

Finally, we use 64-bit doubles and can "only" zoom in by a factor of a few quadrillions times. If this is not enough for you, you can use arbitrary-precision floats to zoom in as far as you wish.

The Mandelbrot set can entertain you for a lifetime.